

IFTS id 107472 TECNICHE DI PROGRAMMAZIONE JAVA, ANDROID E WEB PER L'INDUSTRIA 5.0

Relazione stage presso “ITS Academy Angelo Rizzoli”

Francesco Esposito

INDICE

• 1. Tirocinio	03
- Descrizione dell'Esperienza	
• 2. Attività Svolte	04
- Incarichi Principali	
- Attrezzature Utilizzate	
- Hardware	
- Software	
• 3. Sistema di Gestione Magazzino per Resistenze	07
- Presentazione del Progetto	
- Analisi Funzionale	
- Stack Tecnologico	
- Organizzazione e implementazione del Progetto	
• 4. Database	09
- Modello Entità-Relazione (E-R)	
- entità, attributi e relazioni	
- implementazione MySQL	
- viste del database	
• 5. Backend	16
- progetto SprinBoot	
- Struttura del Progetto	
- Architettura del Backend	
- Accesso al database	
- configurazione	
- Data Transfer Objects (DTO)	
- Controller	
- Servizi	
- Gestione delle missioni	
- Gestione delle eccezioni	
• 6. Frontend	28
- Struttura e layout delle pagine	
- Stile CSS	
- Interazioni JavaScript	
- Caratteristiche principali	
• 7. Conclusioni	33

Il Tirocinio

Il tirocinio si è protratto per circa due mesi, dal 29 aprile al 9 luglio e si è svolto presso la sede dell'ITS Rizzoli, situata in Via Amoretti 78 a Milano, un istituto riconosciuto per la sua eccellenza nella formazione tecnica e professionale.

L'esperienza è stata supervisionata dal professor Carlo Alberto Bassi, un libero professionista specializzato in Data Scientist, Ricercatore Senior in R&I. Attualmente coinvolto in progetti di ricerca a livello locale ed europeo, principalmente occupandosi della parte di analisi e mining dei dati. Inoltre, è anche professore di Analisi dei Dati, Ingegneria dei Dati e Business Intelligence a livello universitario.

Il professor Carlo Alberto Bassi ha ricoperto il ruolo di tutor con grande professionalità, offrendo un supporto costante e mirato durante tutto il percorso. Pur lasciando un'ampia autonomia operativa, indispensabile per favorire l'iniziativa personale e il senso di responsabilità, il professor Bassi è intervenuto con indicazioni precise e chiarimenti tecnici. Questi momenti di confronto, spesso brevi ma estremamente efficaci, hanno riguardato soprattutto la scelta delle tecnologie più appropriate e le migliori pratiche per il loro utilizzo.

Questo periodo ha rappresentato un'occasione significativa per immergersi in un contesto lavorativo, mettendo alla prova le conoscenze teoriche apprese al corso di formazione e sviluppando abilità pratiche.

Oltre agli aspetti tecnici, il tirocinio ha contribuito a sviluppare competenze trasversali fondamentali per il mondo del lavoro. La gestione del tempo, la capacità di risolvere problemi complessi e l'adattabilità a contesti dinamici.

Le Attività

Gli Incarichi

Le attività che hanno caratterizzato questo tirocinio sono state:

- **Analisi dei requisiti:**

Identificazione delle funzionalità principali del sistema, come la gestione delle resistenze, l'elaborazione dei barcode e il controllo delle posizioni nel magazzino tramite schemi.

- **Progettazione del database:**

Creazione dello schema MySQL per la definizione della struttura dati e la sua persistenza.

- **Sviluppo del backend:**

Implementazione della logica applicativa usando il framework Spring Boot, strutturando il progetto in un'architettura a tre livelli.

- **Sviluppo del frontend:**

Realizzazione dell'interfaccia web attraverso la creazione di molteplici pagine HTML con la definizione di script JavaScript per implementare la logica applicativa lato frontend.

- **Testing e debug:**

Esecuzione di test manuali per verificare il sistema e risoluzione dei bug individuati.

- **Riunioni e feedback:**

Partecipazione e incontri con il professor Bassi per discutere i progressi e pianificare le attività successive.

Le Attrezzature

Durante il tirocinio, svolto presso la sede dell'ITS Rizzoli a Milano sotto la supervisione del professor Carlo Bassi, ho utilizzato le seguenti attrezzature hardware e software per lo sviluppo di un programma.

Hardware

Computer fornito dall'ITS Rizzoli, HP: Un computer HP messo a disposizione dall'istituto, dotato di un processore Intel Core i5, 16 GB di RAM e sistema operativo Windows 10. Questo hardware ha garantito prestazioni adeguate allo sviluppo, il debugging e il testing dell'applicazione.

Rete dell'ITS Rizzoli: La connessione Wi-Fi e cablata della sede di Via Amoretti 78 ha fornito accesso a internet e a risorse condivise, facilitando la collaborazione e l'uso di strumenti online.

Software

- **Visual Studio Code:**

Editor di codice sorgente open-source, utilizzato per lo sviluppo dell'applicazione Java con Spring Boot e per la creazione dell'interfaccia web con HTML, CSS e JavaScript. Le sue estensioni, come Spring Boot Tools e Java Extension Pack, hanno supportato il debugging e la gestione del codice.

- **MySQL:**

Sistema di gestione di database relazionali open-source, impiegato per creare e gestire il database del progetto.

- **MySQL Workbench:**

Strumento grafico per la progettazione dello schema del database, l'esecuzione di query SQL e la gestione dei dati, offrendo un'interfaccia intuitiva per le operazioni sul database.

- **Git:**

Sistema di controllo di versione distribuito, utilizzato per tracciare le modifiche al codice sorgente. Un repository remoto su GitHub è stato impiegato per salvare e condividere il lavoro.

- **Postman:**

Piattaforma per lo sviluppo e il testing di API, utilizzata per verificare i servizi RESTful implementati con Spring Boot, garantendo il corretto funzionamento delle richieste e risposte.

- **Maven:**

Strumento di automazione per progetti Java, impiegato per gestire le dipendenze (es. librerie Spring Boot) e automatizzare la compilazione e il packaging dell'applicazione.

- **Google Chrome:**

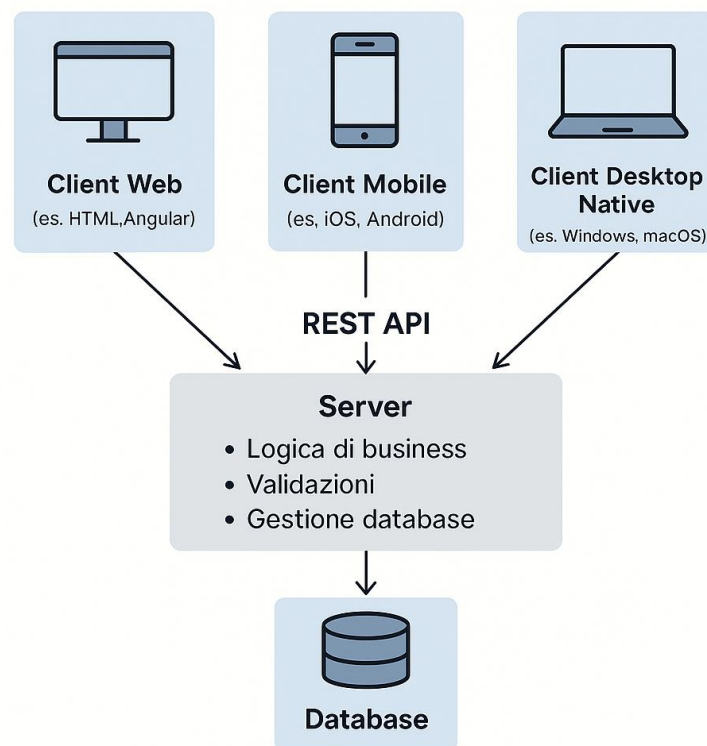
Browser web utilizzato per testare l'interfaccia utente dell'applicazione, verificarne la compatibilità e utilizzare gli strumenti di sviluppo integrati per il debugging del frontend.

Sistema di Gestione Magazzino per Resistenze

Presentazione

Il progetto consisteva nello sviluppo di un applicativo Client-Server per la gestione delle fasi di stoccaggio e picking di un ipotetico magazzino di componentistica elettronica.

Per applicativo client server si intende una soluzione software dove la logica di business è principalmente implementata lato server ed esposta attraverso API HTTP(REST), e dove i clients possono essere implementati su diverse piattaforme (WEB, mobile, Desktop native) e sono responsabili di consumare le funzionalità esposte dall'API e fornire all'utente finale un'interfaccia grafica per interagire con i dati della applicazione.



Analisi Funzionale

Il progetto si basa sui seguenti punti principali:

- **Gestione della struttura del magazzino:** il nostro magazzino è stato modellato con i seguenti parametri “corsia, colonna, ripiano” che ci permetterà di identificare univocamente la posizione del materiale.

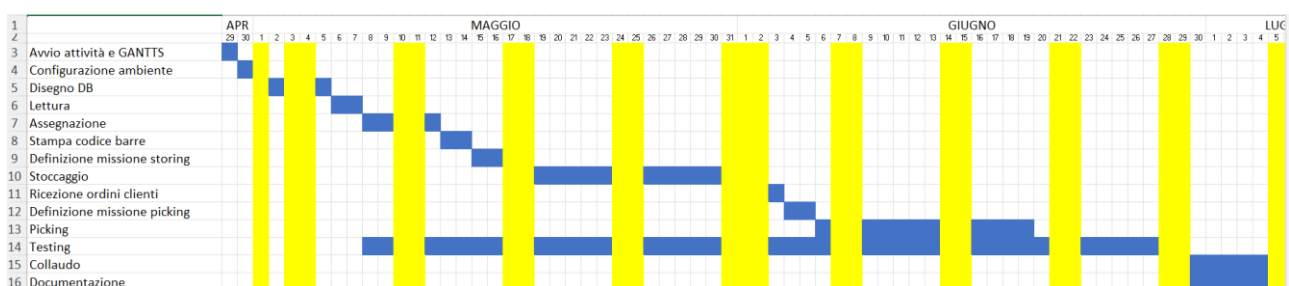
- **Gestione delle resistenze:** Censimento di un articolo attraverso un barcode univoco rispetto alla sua tipologia.
- **Riconoscimento tramite barcode:** inserendo un articolo attraverso il barcode andiamo a recuperare le caratteristiche dell'articolo dall'anagrafica delle resistenze andando a creare un nuovo record di movimento nella tabella dei movimenti.
- **Gestione automatizzata delle posizioni:** Il sistema sceglie automaticamente la migliore posizione utilizzando il criterio della massima occupazione del magazzino. Se la posizione designata per un articolo è già occupata totalmente, il sistema trova automaticamente una posizione alternativa disponibile.
- **Movimenti di entrata e uscita:** il software gestisce il flusso di carico e di scarico di magazzino attraverso l'identificazione puntuale dell'elemento attraverso un codice interno univoco per ogni singola confezione.
Questa scelta progettuale si basa sul principio di minimizzare possibili errori nel trattamento di un genere di prodotti facili da confondere tra loro.
- **Conferma dei movimenti:** Oltre alla fase di ingresso e uscita è prevista una fase di conferma, l'utente deve inserire l'ID codice interno e l'ID della locazione per indicare che l'articolo è stato effettivamente inserito o prelevato dalla corretta locazione.

Stack tecnologico

- Frontend: HTML e JavaScript.
- Backend: Java e Spring Boot
- Persistenza: MySQL

Organizzazione del progetto

Per raggiungere questi obiettivi, il team ha deciso di pianificare le attività giornaliere con un diagramma di Gantt, per coordinare il lavoro e rispettare le scadenze.



Implementazione

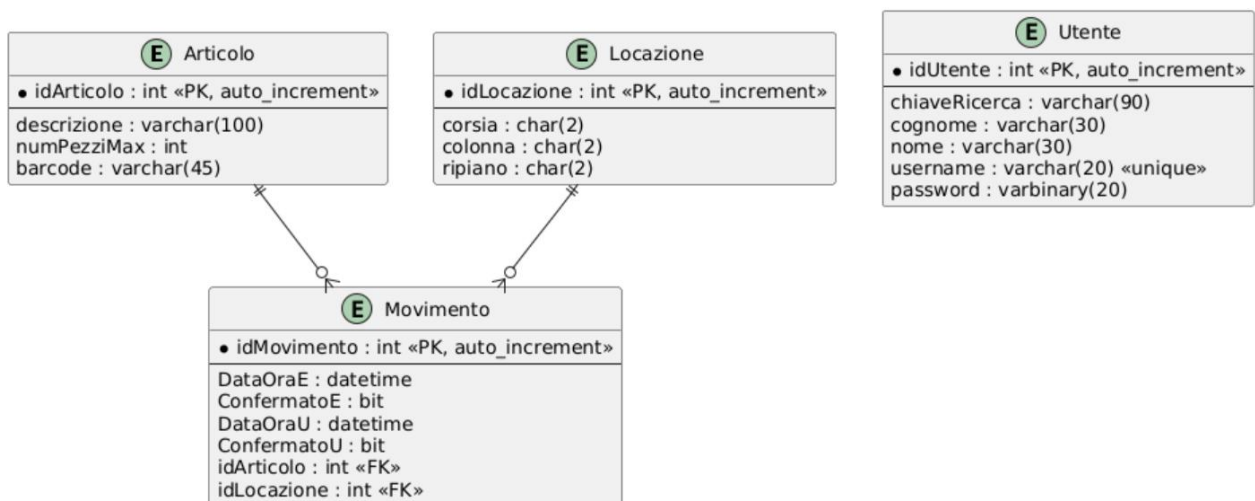
In questa sezione di documento andremo ad analizzare le scelte specifiche implementate per la realizzazione del progetto, in particolare ci concentreremo sulle tre aree principali del progetto: database, backend e frontend cercando di motivare le nostre scelte implementative. Tali scelte sono state fatte tenendo conto dei tempi di sviluppo disponibili e della natura didattica del progetto.

Database

Modello E-R

Per la progettazione del database magazzinoresistenza, è stato sviluppato un modello Entità-Relazione (E-R) per definire in modo strutturato le entità principali, i loro attributi e le loro relazioni, garantendo una gestione efficace delle operazioni di magazzino.

Dopo un'analisi dei requisiti, considerando anche la possibilità di futuri ampliamenti, si è scelto di limitare il modello a quattro entità fondamentali: Articoli, Locazioni, Movimenti e Utenti. Questa struttura riflette le necessità principali del sistema, ossia la gestione degli articoli, delle loro posizioni nel magazzino, dei movimenti di entrata/uscita, delle loro conferme e in uno sviluppo futuro degli utenti autorizzati a operare sul database.



Entità e Attributi:

- **Articoli:**

Rappresenta gli articoli stoccati nel magazzino.

- **Attributi:**

- idArticolo: Chiave primaria, di tipo intero auto-incrementale, che garantisce unicità del record e facilita la gestione di essi.
 - descrizione: varchar(100), descrive l'articolo (es. "Resistenza 10kOhm").
 - numPezziMax: di tipo intero, indica il numero massimo di pezzi che possono essere stoccati per quell'articolo.
 - barcode: varchar(45), codice univoco per identificare l'articolo, utile per ricerche rapide o scansioni.

- **Locazioni:**

Rappresenta le posizioni fisiche nel magazzino, organizzate in corsie, colonne e ripiani.

- **Attributi:**

- idLocazione: Chiave primaria, intero auto-incrementante, scelto per semplificare le relazioni e supportare futuri ampliamenti.
 - corsia: char(2), identifica la corsia (es. "01").
 - colonna: char(2), identifica la colonna (es. "02").
 - ripiano: char(2), identifica il ripiano (es. "03").
 - La combinazione di corsia, colonna e ripiano funge da identificativo univoco, ma si è preferito utilizzare idLocazione per maggiore compattezza.

- **Movimenti:**

Registra i movimenti di entrata e uscita degli articoli nelle locazioni specifiche.

- **Attributi:**

- idMovimento: Chiave primaria, intero auto-incrementante.
 - DataOraE: Datetime, data e ora dell'entrata (nullable, per consentire movimenti non ancora confermati).

- ConfermatoE: bit, nullable, indica se l'entrata è stata confermata
- DataOraU: datetime, nullable, data e ora dell'uscita.
- ConfermatoU: bit, nullable, indica se l'uscita è stata confermata (opzionale).
- idArticolo: Chiave esterna, riferimento ad articoli(idArticolo), per collegare il movimento a un articolo specifico.
- idLocazione: Chiave esterna, riferimento a locazioni(idLocazione), per indicare la posizione del movimento.

- **Utenti:**

Gestisce gli utenti autorizzati a interagire con il sistema, come operatori o amministratori del magazzino.

- **Attributi:**

- idUtente: Chiave primaria, intero auto-incrementante.
- chiaveRicerca: varchar(90), per ricerche rapide degli utenti (es. combinazione di nome e cognome).
- cognome: varchar(30), cognome dell'utente.
- nome: varchar(30), nome dell'utente.
- username: varchar(20), identificativo univoco con una unique constraint per evitare duplicati.
- password: varbinary(20), password crittografata per garantire la sicurezza.

- **Relazioni:**

- **Registra (Articolo-Movimento):**

- Un Articolo può essere coinvolto in più Movimenti (cardinalità 1: N).
- Un Movimento si riferisce a un solo Articolo (N:1).
- La relazione è implementata tramite la chiave esterna idArticolo nella tabella movimenti, che assicura che ogni movimento sia associato a un articolo esistente.

- **Registra (Locazione-Movimento):**

- Una Locazione può essere associata a più Movimenti (1: N).

- Un Movimento si riferisce a una sola Locazione (N:1).
- La relazione è implementata tramite la chiave esterna idLocazione nella tabella movimenti, garantendo che ogni movimento sia collegato a una locazione valida.
- Utente: Nel modello attuale, l'entità Utente non presenta relazioni dirette con altre entità, poiché non ci sono chiavi esterne che la collegano a movimenti, articoli o locazioni. Questa scelta è stata fatta per mantenere il modello semplice, ma consente futuri ampliamenti, come l'aggiunta di una chiave esterna idUtente in movimenti per tracciare chi registra un movimento.

MySQL

Il database magazzinoresistenza è stato creato e gestito utilizzando MySQL, un sistema di gestione di database relazionale open source noto per la sua affidabilità, semplicità e prestazioni elevate. MySQL è stato scelto per la sua compatibilità con le esigenze del progetto e per la facilità di configurazione in un ambiente locale. Le principali funzionalità di MySQL utilizzate includono:

- Chiavi primarie ed esterne: Supportano l'integrità referenziale, essenziale per le relazioni tra articoli, locazioni e movimenti. Le chiavi esterne in movimenti impediscono l'inserimento di dati non validi.
- Tipi di dati: MySQL offre tipi come INT, VARCHAR, CHAR, DATETIME, BIT e VARBINARY, utilizzati per rappresentare gli attributi delle tabelle (es. barcode come VARCHAR(45), password come VARBINARY(20)).
- Viste: Le viste vw_articoli_presenti e vw_occupazione_locazioni sono state create per semplificare query complesse, fornendo dati aggregati senza duplicazione.
- Indicizzazione: Le chiavi primarie creano automaticamente indici, migliorando le prestazioni delle query su idArticolo, idLocazione.

Il database è stato configurato e gestito direttamente in localhost utilizzando il server MySQL installato localmente. La creazione del database è stata eseguita tramite il client MySQL con il seguente comando:

```
2 • create database if not exists magazzinoresistenza;
3 • use magazzinoresistenza;
```

Le tabelle sono state create eseguendo manualmente gli script SQL forniti, che definiscono la struttura di articoli, locazioni, movimenti e utenti, insieme alle chiavi primarie, esterne e alla constraint di unicità su username. Le viste sono state aggiunte successivamente per supportare le analisi operative. Di seguito, degli esempi dello script utilizzato per creare le tabelle:

```
4 • ⊖ create table `articoli` (  
5     `idArticolo` int not null auto_increment,  
6     `descrizione` varchar(100) not null,  
7     `numPezziMax` int not null,  
8     `barcode` varchar(45) not null,  
9     primary key(`idArticolo`)  
10 );  
  
11 • ⊖ create table `locazioni` (  
12     `idLocazione` int not null auto_increment,  
13     `corsia` char(2) not null,  
14     `colonna` char(2) not null,  
15     `ripiano` char(2) not null,  
16     primary key(`idLocazione`)  
17 );  
  
19 • ⊖ create table `movimenti` (  
20     `idMovimento` int not null auto_increment,  
21     `DataOraE` datetime DEFAULT NULL,  
22     `ConfermatoE` bit(1) DEFAULT NULL,  
23     `DataOraU` datetime DEFAULT NULL,  
24     `ConfermatoU` bit(1) DEFAULT NULL,  
25     `idArticolo` int not null,  
26     `idLocazione` int not null,  
27     `codiceInterno` int not null,  
28     primary key (`idMovimento`),  
29     foreign key (`idArticolo`) REFERENCES `articoli`(`idArticolo`),  
30     foreign key (`idLocazione`) REFERENCES `locazioni`(`idLocazione`)  
31 );  
  
34 • ⊖ create table `utenti` (  
35     `idUtente` int not null auto_increment,  
36     `chiaveRicerca` varchar (90) not null,  
37     `cognome` varchar (30) not null,  
38     `nome` varchar (30) not null,  
39     `username` varchar (20) unique not null,  
40     `password` varbinary (20) not null,  
41     primary key (`idUtente`)  
42 );
```

Le viste sono state create con i seguenti comandi:

```
45 • CREATE
46     ALGORITHM = UNDEFINED
47     DEFINER = `root`@`localhost`
48     SQL SECURITY DEFINER
49 VIEW `magazzinoresistenza`.`vw_articoli_presenti` AS
50     SELECT
51         `magazzinoresistenza`.`movimenti`.`IdLocazione` AS `idlocazione`,
52         `magazzinoresistenza`.`movimenti`.`IdArticolo` AS `idarticolo`,
53         SUM(1) AS `NumeroPezzi`
54     FROM
55         `magazzinoresistenza`.`movimenti`
56     WHERE
57         (`magazzinoresistenza`.`movimenti`.`DataOraU` IS NULL)
58     GROUP BY `magazzinoresistenza`.`movimenti`.`IdLocazione` , `magazzinoresistenza`.`movimenti`.`IdArticolo`;
```

La vista **vw_articoli_presenti**, definita nel database MySQL dello schema **magazzinoresistenza**, ha l'obiettivo di fornire una rappresentazione aggregata degli articoli attualmente presenti in magazzino, organizzati per locazione e tipo di articolo. La vista si basa sulla tabella **movimenti** e restituisce un elenco che include l'identificativo della locazione (**IdLocazione**), l'identificativo dell'articolo (**IdArticolo**) e il numero totale di pezzi (**NumeroPezzi**) per ciascuna combinazione.

La vista esegue le seguenti operazioni:

- **Selezione dei dati:** Estrae **IdLocazione** e **IdArticolo** dalla tabella **movimenti**, calcolando il numero di pezzi tramite l'aggregazione **SUM (1)** per ogni combinazione di locazione e articolo.
- **Filtro:** Include solo i record in cui il campo **DataOraU** (data e ora di uscita) è **NULL**, indicando che gli articoli non sono stati prelevati e sono ancora presenti in magazzino.
- **Raggruppamento:** Aggrega i risultati utilizzando **GROUP BY** su **IdLocazione** e **IdArticolo**, garantendo un conteggio accurato degli articoli per locazione.

```

61 • CREATE
62 ALGORITHM = UNDEFINED
63 DEFINER = `root`@`localhost`
64 SQL SECURITY DEFINER
65 VIEW `magazzinoresistenza`.`vw_occupazione_locazioni` AS
66 SELECT
67     `lo`.`IdLocazione` AS `IdLocazione`,
68     `lo`.`Corsia` AS `Corsia`,
69     `lo`.`Colonna` AS `Colonna`,
70     `lo`.`Ripiano` AS `Ripiano`,
71     `magazzinoresistenza`.`vw`.`NumeroPezzi` AS `NumeroPezzi`,
72     `magazzinoresistenza`.`vw`.`IdArticolo` AS `IdArticolo`
73 FROM
74     (`magazzinoresistenza`.`locazioni` `lo`
75     LEFT JOIN `magazzinoresistenza`.`vw_articoli_presenti` `vw` ON ((`lo`.`IdLocazione` = `magazzinoresistenza`.`vw`.`idlocazione`)))

```

La vista **vw_occupazione_locazioni**, definita nello schema **magazzinoresistenza** del database MySQL, ha l'obiettivo di fornire una panoramica completa dell'occupazione delle locazioni nel magazzino, indicando quali locazioni contengono articoli, la quantità di pezzi per articolo e i dettagli di posizione (corsia, colonna, ripiano). La vista combina i dati della tabella **locazioni** con quelli della vista **vw_articoli_presenti** per mostrare sia le locazioni occupate che quelle vuote.

La vista esegue le seguenti operazioni:

- Selezione dei dati:
 - Dalla tabella **locazioni** (**lo**): Estrae **IdLocazione**, **Corsia**, **Colonna** e **Ripiano**, che definiscono la posizione fisica di ogni locazione nel magazzino.
 - Dalla vista **vw_articoli_presenti** (**vw**): Estrae **NumeroPezzi** (quantità di articoli presenti) e **IdArticolo** (identificativo dell'articolo).
- Join: Utilizza un **LEFT JOIN** per collegare la tabella **locazioni** con la vista **vw_articoli_presenti** tramite il campo **IdLocazione**. Il **LEFT JOIN** garantisce che tutte le locazioni siano incluse, anche quelle senza articoli (dove **NumeroPezzi** e **IdArticolo** saranno **NULL**).
- Risultato: Restituisce una tabella con **IdLocazione**, **Corsia**, **Colonna**, **Ripiano**, **NumeroPezzi** e **IdArticolo** per ogni locazione, indicando l'occupazione attuale.

Progetto Backend

In questa sezione il focus è sulla implementazione dell'API e dell'accesso al database:
il progetto di backend è costituito da due progetti Maven separati:

- **MagazzinoDb:** una Libreria java standard che permette l'accesso al database.
- **Magazzino:** un progetto Spring Boot che implementa le API consumate dal frontend.

Accesso al database

Le funzionalità di accesso al database sono state sviluppate come libreria autonoma dalla logica implementativa per mantenere un alto livello di utilizzabilità, questa libreria è stata sviluppata utilizzando JAVA nativo e JDBC. Questo permetterà l'accesso alle funzionalità del database a tutti gli applicativi Java indipendentemente dal framework utilizzato.

Le classi contenute nel package models sono la rappresentazione delle tabelle del database e le loro proprietà rappresentano le colonne delle tabelle. Questo approccio permette di astrarre la struttura del database utilizzando la OOP (object oriented programming).

- **Articoli:** Rappresenta un articolo con idArticolo, descrizione, numPezziMax e barcode.
- **Locazioni:** Definisce una locazione con idLocazione, corsia, colonna e ripiano.
- **Movimenti:** Registra i movimenti con idMovimento, confermatoE, confermatoU, dataOraE, dataOraU, idArticolo, idLocazione e codiceInterno.
- **Utenti:** Gestisce le informazioni degli utenti, con chiaveRicerca per ricerche rapide.
- **MissioniE e MissioniU:** Rappresentano missioni di entrata e uscita.
- **ViewOccupazioneLocazioni & vw_articoli_presenti :** Viste per monitorare l'occupazione delle locazioni e la presenza degli articoli.

Le classi *DbUtil gestiscono le operazioni CRUD sulla tabella di riferimento tramite JDBC:

- **ArticoliDbUtil:** Operazioni sugli articoli (es. salvaArticolo, recuperaArticoloConBarcode, modificaArticolo, cancellaArticoloConId).
- **LocazioniDbUtil:** Gestione delle locazioni (es. salvaLocazione, recuperLocazioneConId, modificaLocazioneConId, cancellaLocazioniConId).

- **MovimentoDbUtil:** Operazioni sui movimenti, inclusa la generazione automatica di codiceInterno.
- **UtentiDbUtil:** Gestione degli utenti, con crittografia delle password tramite MD5.

Un caso particolare è rappresentato dalle due classi WOccupazioneLocazioneDbUtil e WArticoliPresentiDbUtil che Gestiscono l'accesso ai dati delle viste per l'occupazione delle locazioni e la presenza degli articoli.

Le query utilizzano PreparedStatement per prevenire SQL injection, garantendo la sicurezza delle operazioni sul database. I risultati delle query sono mappati su oggetti Java tramite ResultSet. Per gestire le risorse del database in modo sicuro ed efficiente, tutte le classi *DbUtil utilizzano la costruzione try-with-resources per le connessioni PreparedStatement e ResultSet. Questo approccio assicura la chiusura automatica delle risorse al termine del loro utilizzo, anche in caso di eccezioni, riducendo il rischio di perdite di memoria e migliorando la gestione delle connessioni al database.

Esempio di utilizzo di try-with-resources in ArticoliDbUtil:

```
public Articolo salvaArticolo(Articolo articolo) {
    String queryInserisciArticolo = "insert into articoli (descrizione, numPezziMax, barcode) values(?,?,?)";

    try (PreparedStatement statementSalvaArticolo = connection.prepareStatement(queryInserisciArticolo);) {

        statementSalvaArticolo.setString(parameterIndex:1, articolo.getDescrizione());
        statementSalvaArticolo.setInt(parameterIndex:2, articolo.getNumPezziMax());
        statementSalvaArticolo.setString(parameterIndex:3, articolo.getBarcode());
        int risultato = statementSalvaArticolo.executeUpdate();
        if (risultato > 0) {
            String querySalvaArticolo = "select * from articoli order by idArticolo desc";
            PreparedStatement statementLeggiArticoli = connection.prepareStatement(querySalvaArticolo);
            ResultSet resultQuery = statementLeggiArticoli.executeQuery();

            if (resultQuery.next()) {
                int idArticolo = resultQuery.getInt(columnLabel:"idArticolo");
                articolo.setIdArticolo(idArticolo);
            }
        }
        statementSalvaArticolo.close();
    } catch (Exception e) {
        e.printStackTrace();
    }
    return articolo;
}
```

Il motivo per cui si utilizza try-with-resources è la gestione automatica delle risorse che implementano l'interfaccia AutoCloseable (come PreparedStatement e ResultSet). Questo elimina la necessità di chiudere manualmente le risorse nel blocco finally, riducendo il codice boilerplate e migliorando la leggibilità e l'affidabilità del codice.

Progetto Spring boot

Lo sviluppo delle API è stato realizzato utilizzando Spring Boot, un framework Java che semplifica lo sviluppo di applicazioni robuste e scalabili grazie al suo approccio convention over configuration e alla gestione automatica delle dipendenze.

Questa sezione descrive l'architettura del backend, le tecnologie utilizzate, la struttura del database, i componenti principali del progetto e le funzionalità implementate, con particolare attenzione ai controller, ai servizi e alla gestione delle interazioni con il database.

Spring Boot offre un ambiente di sviluppo semplificato grazie ai seguenti principi:

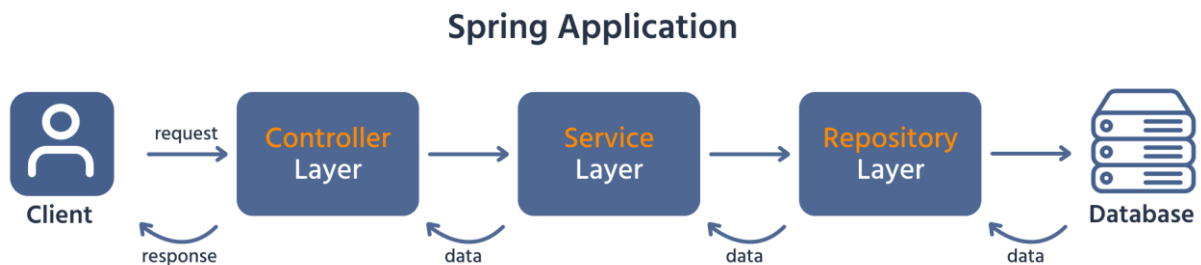
- **Dependency Injection (DI):** Spring Boot gestisce automaticamente le dipendenze tramite l'iniezione, riducendo la necessità di configurazioni manuali.
- **Plain Old Java Object (POJO):** Le entità e i DTO (Data Transfer Object) sono stati implementati come POJO, garantendo semplicità e riutilizzabilità del codice.
- **Spring MVC:** Utilizzato per implementare un'architettura RESTful, con controller che gestiscono le richieste HTTP dal frontend.
- **Convention over Configuration:** Spring Boot fornisce configurazioni predefinite, permettendo un avvio rapido del progetto.

Il progetto è stato inizializzato tramite Spring Initializr e contiene le seguenti dipendenze definite nel pom.xml:

- **spring-boot-starter-web:** Per la gestione delle API REST e il servizio delle risorse statiche (HTML, CSS).
- **mysql-connector-java (8.0.28):** Per la connessione al database MySQL.
- **lombok:** Per ridurre il codice boilerplate tramite annotazioni come `@Getter`, `@Setter`, `@NoArgsConstructor` e `@AllArgsConstructor`.
- **magazzinoDb:** Modulo personalizzato per l'accesso al database, utilizzato come dipendenza nel modulo principale.

Struttura del progetto API

La struttura del progetto segue il pattern architetturale dei 3 layer.



La struttura dei layer è stata implementata dai seguenti package e le loro classi:

- **Controller:** Gestiscono le richieste HTTP e servono le pagine HTML statiche (presentation layer).
- **Service:** Implementano la logica di business (service layer).
- **DbUtil:** Gestiscono le operazioni sul database tramite JDBC (data layer).
- **DTO:** Facilitano il trasferimento dei dati tra i diversi layer.

Controller

Spring Boot permette di designare classi POJO con specifiche annotazioni per essere gestite dal contesto Spring; nel nostro caso la classe **BarcodeController** è stata annotata come **@RestController**: è un'annotazione che indica che una classe è un controller web i cui metodi gestiscono le richieste HTTP e restituiscono direttamente i dati (tipicamente in formato JSON o XML), invece di restituire una vista HTML.

```
@RestController
public class BarcodeController {

    @Autowired
    private BarcodeService barcodeService;

    @PostMapping(value = "/barcode", consumes = "application/json")
    public ResponseEntity<String> aggiungiArticolo(@RequestBody RichiestaInserisciBarcode entity) {

        barcodeService.aggiungiBarcode(entity.getBarcode(), entity.getDescrizione());
        return new ResponseEntity<>(HttpStatus.CREATED);
    }
}
```

La seconda annotazione che incontriamo è quella dell'@**Autowired**, questa annotazione delega al framework Spring di identificare nel contesto un oggetto di tipo **BarcodeService**.

Se un'istanza di quest'oggetto non fosse disponibile nel contesto, la classe BarcodeController fallirebbe nella sua inizializzazione.

La terza annotazione che incontriamo è @**PostMapping** che permette di legare un metodo della classe POJO ad una risorsa HTTP e al verbo POST. Questa annotazione contiene due attributi:

- **value:** contiene il path a cui risponderà questo metodo.
- **consumes:** specifica il tipo di body che il metodo accetterà, in questo caso *application/json*.

La quarta annotazione è @**RequestBody** lega il body ricevuto nella chiamata HTTP con la classe di DTO.

Lo statement di return ritorna un'entità della classe **ResponseEntity**, che è utilizzata per rappresentare una risposta HTTP completa, includendo:

- **Corpo della risposta** (i dati inviati al client, es. un oggetto JSON, una stringa, ecc.).
- **Codice di stato HTTP** (es. 200 OK, 404 Not Found, 500 Internal Server Error).
- **Intestazioni HTTP** (es. Content-Type, Cache-Control, ecc.).

Nel nostro caso ritorniamo una risposta con codice 201(CREATED) e body vuoto.

```
@PostMapping(value = "movimenti/entrata/conferma")
public ResponseEntity<String> confermaEntrata(@RequestParam("codiceInterno") int codiceInterno,
        @RequestParam("idLocazione") int idLocazione) {
    try {
        boolean risultato = barcodeService.confermaMovimentoE(codiceInterno, idLocazione);
        if (risultato == false) {
            return new ResponseEntity<>(body:"Movimento non trovato", HttpStatus.NOT_FOUND);
        }

        return new ResponseEntity<>(body:"Entrata confermata", HttpStatus.OK);
    } catch (DataLayerException e) {
        return new ResponseEntity<>(HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

Un secondo utilizzo vede il metodo **confermaEntrata** essere mappata sulla risorsa: "movimenti/entrata/conferma" e verbo PUT. Nei parametri del metodo incontriamo l'annotazione @**RequestParam**, che lega uno specifico query parameter con un parametro del metodo. Quindi per attivare questo metodo il nostro RestController dovrà ricevere una chiamata del tipo **PUT: /movimenti/entrata/conferma?codiceInterno=10&idLocazione=5**

Nella risposta, la `ResponseEntity` possiamo trovare un body di tipo `Stringa` e un `HTTP STATUS 200` che rappresenta l'avvenuto aggiornamento della risorsa.

In questo caso di eccezione `DataLayerException`, ritorniamo una risposta `HTTP` con `STATUS 500` che rappresenta un errore generico di servizio.

```
@GetMapping(value = "movimenti/entrata", produces = "application/json")
public ResponseEntity<ResponseMissione> recuperaMissioneEntrata() {
    try {
        ArrayList<Missione> missioniEntrata = barcodeService.getMissioni(entrataUscita:true);

        ResponseMissione response = new ResponseMissione();

        response.setMissioni(new ArrayList<>());
        for (Missione missioneEntrata : missioniEntrata) {
            it.francesco.magazzino.controllers.dto.Missione missione = new it.francesco.magazzino.controllers.dto.Missione(
                missioneEntrata.getBarcode(),
                missioneEntrata.getDescrizione(),
                missioneEntrata.getCodiceInterno(),
                missioneEntrata.getIdLocazione(),
                missioneEntrata.getCorsia(),
                missioneEntrata.getColonna(),
                missioneEntrata.getRipiano());
            response.getMissioni().add(missione);
        }
        response.setRecordTotali(missioniEntrata.size());
        return new ResponseEntity<>(response, HttpStatus.OK);
    } catch (DataLayerException e) {
        return new ResponseEntity<>(HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

Come si può vedere nell'immagine sopra stiamo mappando la risorsa: **movimenti/entrata** con il vero `GET` che permette di recuperare una lista di risorse. Il nostro oggetto di risposta contiene una struttura complessa con una `Array` dei movimenti di entrata e il numero dei movimenti.

```
1  {
2    "recordTotali": 46,
3    "missioni": [
4      {
5        "barcode": "1234",
6        "descrizione": "resistenza 100 ohm",
7        "codiceInterno": 0,
8        "idLocazione": 1,
9        "corsia": "01",
10       "colonna": "01",
11       "ripiano": "01"
12     },
```

È da notare la presenza dell'attributo `produces`, che definisce il tipo di body prodotto dal metodo (`application/json`).

Questa è la lista di tutte le funzionalità esposte del controller:

- **WebController** (@Controller):

- *Serve pagine HTML statiche dalla directory src/main/resources/static.*
- Endpoint:
 - GET /: Serve index.html, la homepage dell'applicazione.
 - GET /inserisciArticoli.html: Serve la pagina per l'inserimento di nuovi articoli.
 - GET /inserisciBarcode.html: Serve la pagina per l'inserimento del barcode.
 - GET /confermaIngresso.html: Serve la pagina per confermare l'ingresso degli articoli.
 - GET /confermaUscita.html: Serve la pagina per confermare l'uscita degli articoli.
 - GET /prelevaArticolo.html: Serve la pagina per il prelievo degli articoli.

BarcodeController (@RestController):

- *Gestisce le API REST per le operazioni di magazzino.*
- Endpoint:
 - POST /barcode: Aggiunge un articolo con barcode e descrizione.
 - POST /movimenti/entrata: Trova una locazione per un articolo in base al barcode.
 - PUT /movimenti/entrata/conferma: Conferma un movimento di entrata verificando il codice interno e l'ID della locazione.
 - GET /movimenti/entrata: Recupera l'elenco delle missioni di entrata non confermate.
 - PUT /movimenti/uscita: Esegue un movimento di uscita per un articolo.
 - PUT /movimenti/uscita/conferma: Conferma un movimento di uscita.
 - GET /movimenti/uscita: Recupera l'elenco delle missioni di uscita non confermate.

Servizi

Spring Boot permette di designare classi POJO con specifiche annotazioni per essere gestite dal contesto Spring; nel nostro caso la classe **BarcodeService** è stata annotata come **@Service** che è una specializzazione dell'annotazione **@Component** utilizzata per contrassegnare una classe come un bean di servizio nel contesto dell'applicazione. Indica che la classe contiene la logica di business dell'applicazione, separandola da altre responsabilità come la gestione delle richieste HTTP (controller) o l'accesso ai dati (repository o utilità di database).

```
@Service
public class BarcodeService {

    @Autowired
    private ArticoliDbUtil articoliDbUtil;
    @Autowired
    private WOccupazioneLocazioneDbUtil wLocationDisponibiliDbUtil;
    @Autowired
    private WArticoliPresentiDbUtil wArticoliPresentiDbUtil;
    @Autowired
    private MovimentoDbUtil movimentoDbUtil;
    @Autowired
    private LocazioniDbUtil locazioniDbUtil;

    public void aggiungiBarcode(String barcode, String descrizione) {
        Articolo articolo = new Articolo();
        articolo.setBarcode(barcode);
        articolo.setDescrizione(descrizione);
        articolo.setNumPezziMax(numPezziMax:10);
        articoliDbUtil.salvaArticolo(articolo);
    }
}
```

Si possono notare l'iniezione di diverse classi DbUtil definite nella libreria magazzinoDb che nativamente non utilizza le annotazioni Spring Boot ma come spiegheremo nella sezione configurazione è possibile renderla utilizzabile anche in un contesto Spring Boot.

Ultimo punto è da notare che la firma del metodo crea aggiungiBarcode non ha la stessa classe dell'equivalente metodo del controller; questo anche se può sembrare una non necessaria complicazione aggiunge livelli di flessibilità tra i layer dell'applicazione.

La classe **BarcodeService** (**@Service**) contiene le seguenti implementazioni della logica di business:

- **aggiungiBarcode:** Salva un articolo nel database.
- **trovaLocazione:** Assegna una locazione disponibile in base al barcode e alla capacità.

- **confermaMovimentoE e confermaMovimentoU:** Confermano i movimenti aggiornando lo stato e il timestamp.
- **eseguiUscita:** Registra un movimento di uscita.
- **getMissioni:** Recupera missioni di entrata o uscita.

Configurazione

La classe `ConfigClientDb` è annotata con l'annotazione `@Configuration`, questa annotazione definisce una classe che verrà eseguita in inizializzazione del contesto Spring e permette di istanziare Bean anche da classi non annotate con annotazioni Spring Boot.

La seconda annotazione che incontriamo `@Value` permette di assegnare un valore a un parametro della classe; per esempio, il valore `db.host` verrà assegnato al parametro `host`. Tali valori sono contenuti nel file `application.properties`.

Questo approccio permette di disaccoppiare la programmazione dalla configurazione dell'applicativo, per esempio, potremmo dover cambiare l'host del database senza dover toccare il codice.

```
@Configuration

public class ConfigClientDb {
    @Value("${db.host}")
    private String host;
    @Value("${db.dbName}")
    private String dbName;
    @Value("${db.username}")
    private String username;
    @Value("${db.password}")
    private String password;
    @Value("${db.portNumber}")
    private int portNumber;

    @Bean
    public ArticoliDbUtil articoliDbUtil() {
        System.out.println(host + dbName + username + password + portNumber);
        return new ArticoliDbUtil(host, dbName, username, password, portNumber);
    }
}
```

L'ultima annotazione `@Bean` in Spring Boot è utilizzata per dichiarare un metodo che produce un bean da registrare nel contesto dell'applicazione di Spring.

Nel nostro caso verrà creato un Bean di tipo **ArticoloDbUtil** che potrà essere utilizzato dal service come precedentemente mostrato.

DTO (Data Transfer Object)

```
@NoArgsConstructor
@AllArgsConstructor
@Setter
@Getter
public class RichiestaMovimentazioneBarcode {
    @JsonProperty("barcode")
    private String barcode;
}
```

La classe **RichiestaMovimentazioneBarcode** presenta diverse annotazioni:

- **@NoArgsConstructor:** questa annotazione fa parte dell'annotazioni lombok e istruisce il plug-in lombok a costruire la classe RichiestaMovimentazioneBarcode con un costruttore vuoto.
- **@AllArgsConstructor:** questa annotazione fa parte dell'annotazioni lombok e istruisce il plug-in lombok a costruire la classe RichiestaMovimentazioneBarcode con un costruttore con tutti i parametri.
- **@Setter:** questa annotazione fa parte dell'annotazioni lombok e istruisce il plug-in lombok a costruire la classe RichiestaMovimentazioneBarcode con un metodo di set per ogni parametro della classe.
- **@Getter:** questa annotazione fa parte dell'annotazioni lombok e istruisce il plug-in lombok a costruire la classe RichiestaMovimentazioneBarcode con un metodo di get per ogni parametro della classe

La classe generata può essere trovata nella directory target.classes

```
public class RichiestaMovimentazioneBarcode {
    @JsonProperty("barcode")
    private String barcode;

    @Generated
    public RichiestaMovimentazioneBarcode() {
    }

    @Generated
    public RichiestaMovimentazioneBarcode(final String barcode) {
        this.barcode = barcode;
    }

    @JsonProperty("barcode")
    @Generated
    public void setBarcode(final String barcode) {
        this.barcode = barcode;
    }

    @Generated
    public String getBarcode() {
        return this.barcode;
    }
}
```

L'ultima annotazione è **@JsonProperty** che permette di mappare una proprietà di un oggetto json con una proprietà della classe.

Nel mio progetto sono utilizzati i seguenti DTO:

- **RichiestaMovimentazioneBarcode:** Contiene il barcode per le richieste di movimentazione.
- **RichiestaInserisciBarcode:** Include barcode e descrizione per l'aggiunta di articoli.
- **ResponseGetLocation:** Restituisce dettagli sulla locazione (es. idLocazione, corsia, colonna, ripiano).
- **ResponseMissione:** Contiene una lista di missioni e il numero totale di record.
- **Missione:** Rappresenta una missione con barcode, descrizione, codiceInterno, idLocazione, corsia, colonna e ripiano.
- **LocationInfo:** Trasferisce informazioni sulle locazioni internamente.

Gestione dell'eccezioni

La gestione delle eccezioni è implementata tramite la classe personalizzata

DataLayerException, utilizzata per segnalare errori relativi alle operazioni sul database.

Questa classe estende **Exception** e fornisce un messaggio di default ("Errore data layer") per identificare problemi nel livello di accesso ai dati.

```
3 public class DataLayerException extends Exception {
4
5     public DataLayerException () {
6         super(message: "Errore data layer");
7     }
8
9 }
10
```

Utilizzo di DataLayerException

La classe è utilizzata in BarcodeService e MovimentoDbUtil per gestire errori nelle operazioni:

- **In BarcodeService:**
 - Metodi come trovaLocazione, confermaMovimentoE, confermaMovimentoU, eseguiUscita e getMissioni dichiarano throws DataLayerException per propagare errori dal livello di accesso al database (es. MovimentoDbUtil, ArticoliDbUtil) al chiamante.
 - Esempio: In trovaLocazione, un errore nella query per recuperare un articolo o una locazione (es. SQLException) viene catturato e rilanciato come DataLayerException.
- **In MovimentoDbUtil:**
 - Metodi come salvaMovimento, recuperaMovimentoConId, modificaMovimentoConId, cancellaMovimentoConId, cercaMovimentoPerCodiceInterno, missioneEntrata, missioneUscita e creaCodiceInterno catturano SQLException e lanciano DataLayerException.
 - Esempio: In salvaMovimento, un errore SQL (es. violazione di vincoli o connessione fallita) viene intercettato e trasformato in DataLayerException.

Un esempio di utilizzo della classe DataLayerException:

```
public Movimento recuperaMovimentoConId(int idMovimento) throws DataLayerException {
    String queryRecuperaMovimento = "SELECT * FROM movimenti WHERE idMovimento = ?";
    try (PreparedStatement statementRecuperaMovimento = connection.prepareStatement(queryRecuperaMovimento)) {
        statementRecuperaMovimento.setInt(parameterIndex:1, idMovimento);
        try (ResultSet resultQuery = statementRecuperaMovimento.executeQuery()) {
            if (resultQuery.next()) {
                boolean confermatoE = resultQuery.getBoolean(columnLabel:"confermatoE");
                boolean confermatoU = resultQuery.getBoolean(columnLabel:"confermatoU");
                LocalDateTime dataOraE = resultQuery.getObject(columnLabel:"dataOraE", type:LocalDateTime.class);
                LocalDateTime dataOraU = resultQuery.getObject(columnLabel:"dataOraU", type:LocalDateTime.class);
                int idArticolo = resultQuery.getInt(columnLabel:"idArticolo");
                int idLocazione = resultQuery.getInt(columnLabel:"idLocazione");
                int codiceInterno = resultQuery.getInt(columnLabel:"codiceInterno");
                return new Movimento(idMovimento, confermatoE, confermatoU, dataOraE, dataOraU, idArticolo,
                    idLocazione, codiceInterno);
            } else {
                return null;
            }
        }
    }
} catch (SQLException e) {
    e.printStackTrace();
    throw new DataLayerException();
}
```

Frontend

Il frontend di MagazzinoResistenze è un'interfaccia utente essenziale e funzionale, progettata per consentire agli utenti di gestire il magazzino attraverso operazioni come l'inserimento di articoli, la registrazione di movimenti di entrata e uscita, e la loro conferma. È composto da pagine HTML statiche servite dal WebController di Spring Boot, stilizzate tramite due file CSS; index.css per la homepage e common.css per tutte le altre pagine operative.

Le interazioni dinamiche sono gestite tramite JavaScript utilizzando l'API fetch per inviare richieste HTTP asincrone agli endpoint REST del BarcodeController.

Le risposte del server sono elaborate in modo asincrono tramite il metodo .then, che gestisce i dati di successo, con il metodo .catch catturiamo eventuali errori (es. problemi di rete o risposte non valide), garantendo un feedback chiaro all'utente attraverso messaggi visualizzati nell'interfaccia.

Struttura e Layout delle Pagine

Il frontend include sei pagine HTML statiche, ciascuna con un flusso specifico nella gestione del magazzino:

- index.html: Homepage che funge da menu principale, con link alle altre pagine.
- inserisciArticoli.html: Per aggiungere nuovi articoli inserendo descrizione e barcode.
- inserisciBarcode.html: Per avviare un movimento di entrata fornendo un barcode.
- confermaIngresso.html: Per confermare i movimenti di entrata tramite codice interno e ID locazione.
- prelevaArticolo.html: Per avviare un movimento di uscita inserendo il codice interno.
- confermaUscita.html: Per confermare i movimenti di uscita con codice interno e ID locazione.

Tutte le pagine seguono un layout standardizzato:

- Intestazione (<div class="header-section">): Contiene un titolo descrittivo e, tranne in index.html, un link per tornare alla homepage.
- Linea orizzontale (<div class="horizontal-line">): Separa visivamente l'intestazione dal contenuto.

- Form (<div class="form-container">): Include campi di input (es. per barcode, descrizione, codice interno, ID locazione) e pulsanti per attivare operazioni, organizzati in righe (<div class="form-row">).
- Area messaggi (<div class="message-container">): Mostra feedback come conferme o errori, con testo evidenziato in grassetto per "Messaggio:".

Stile CSS

Il frontend utilizza due file CSS per garantire un design coerente:

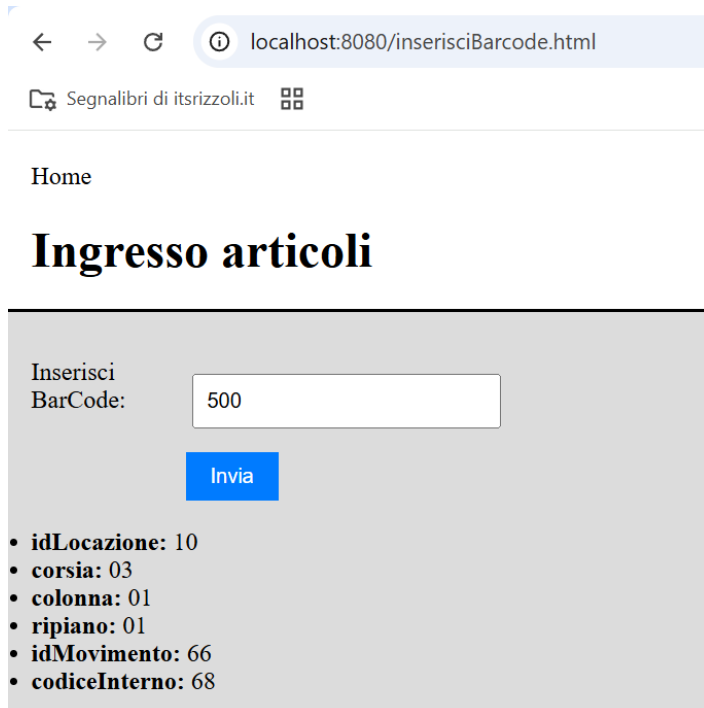
- **index.css** (specifico per index.html):
 - Reset di base: Rimuove margini e padding predefiniti (* { margin: 0; padding: 0; box-sizing: border-box; }) per un controllo preciso del layout.
 - Body: Sfondo grigio chiaro (gainsboro), testo nero, font Times New Roman.
 - Intestazione: Sfondo bianco, padding verticale di 20px, titolo spostato a destra di 15px.
 - Linea orizzontale: Linea nera di 2px con margine inferiore di 30px per separare i contenuti.
 - Link: Disposti su righe separate nel <div class="links-container">, senza sottolineatura, colore nero, con effetto hover viola per interattività.

- index.css:



- **common.css** (utilizzato da `inserisciArticoli.html`, `inserisciBarcode.html`, `confermaIngresso.html`, `prelevaArticolo.html`, `confermaUscita.html`):
 - Reset di base: Come in `index.css`, elimina margini e padding predefiniti e usa `box-sizing: border-box` per includere padding e bordi nel calcolo delle dimensioni.
 - Body: Sfondo grigio chiaro (gainsboro), testo nero, font Times New Roman, garantendo coerenza visiva con la homepage.
 - Intestazione (`<div class="header-section">`): Sfondo bianco, padding verticale di 20px, titolo spostato a destra di 15px. Il link alla homepage è allineato a sinistra con margine di 15px, senza sottolineatura, colore nero, e effetto hover viola.
 - Linea orizzontale (`<div class="horizontal-line">`): Linea nera di 2px con margine inferiore di 30px, identica a `index.css`.
 - Form (`<div class="form-container">`):
 - Pulsanti (`<button>`):
 - Effetto hover che scurisce il blu (`#0056b3`) per interattività.
 - Area messaggi (`<div class="message-container">`): Allineata a sinistra con margine di 15px, coerente con il layout generale.

- **common.css:**



← → ↻ ⓘ localhost:8080/inserisciBarcode.html

🔖 Segnalibri di itsrizzoli.it 🗖

Home

Ingresso articoli

Inserisci BarCode:

Invia

- idLocazione: 10
- corsia: 03
- colonna: 01
- ripiano: 01
- idMovimento: 66
- codiceInterno: 68

Interazioni JavaScript

Il frontend utilizza JavaScript con l'API fetch per comunicazioni asincrone con il backend, inviando richieste HTTP agli endpoint REST del BarcodeController. Le interazioni principali sono:

- **Inserimento articoli (inserisciArticoli.html):**
L'utente inserisce descrizione e barcode in un form. Cliccando "Salva", la funzione `inviaDati()` invia una richiesta POST a `/barcode` con un payload JSON (`{ descrizione, barcode }`). Se la richiesta ha successo (`response.ok`), mostra "Resistenza salvata" per 2 secondi; altrimenti, visualizza un errore.
- **Movimenti di entrata (inserisciBarcode.html):**
L'utente inserisce un barcode e clicca "Invia". Una richiesta POST a `/movimenti/entrata` restituisce un JSON con dettagli della locazione (es. `idLocazione`, `corsia`, `colonna`, `ripiano`, `idMovimento`, `codiceInterno`), visualizzati in una lista HTML ordinata all'interno del `<div id="message">`.
- **Conferma entrata (confermaIngresso.html):**
L'utente fornisce codice interno e ID locazione, inviando una richiesta PUT a `/movimenti/entrata/conferma` con parametri di query (`codiceInterno`, `idLocazione`). Il risultato testuale (es. conferma o errore) viene mostrato nel `<p id="result">`.
- **Movimenti di uscita (prelevaArticolo.html):**
L'utente inserisce il codice interno e invia una richiesta PUT a `/movimenti/uscita` con il parametro di query `codiceInterno`. Il risultato testuale viene visualizzato nel `<div id="result">`.
- **Conferma uscita (confermaUscita.html):**
Simile a `confermaIngresso.html`, invia una richiesta PUT a `/movimenti/uscita/conferma` con parametri di query `codiceInterno` e `idLocazione`, mostrando il risultato testuale nel `<p id="result">`.

Le richieste utilizzano JSON per i dati in ingresso (POST) e parametri di query per le conferme (PUT), con gestione dei risultati tramite promesse (`then/catch`) per aggiornare dinamicamente l'interfaccia.

```

32     document.getElementById("submitBtn").addEventListener("click", function () {
33         event.preventDefault();
34         const barcode = document.getElementById("barcode").value;
35
36         const payload = { barcode: barcode };
37         console.log(payload);
38
39         fetch(`movimenti/entrata`, {
40             method: "POST",
41             headers: {
42                 'Accept': 'application/json',
43                 'Content-Type': 'application/json'
44             },
45             body: JSON.stringify(payload),
46         })
47     })

```

In questa sezione di codice di JavaScript troviamo la definizione di un event handler per il componente con id submitBtn ed evento click. Alla chiamata dell'evento la funzione costruirà un payload json contenente il codice barcode inserito dall'utente nel form e utilizzerà la libreria fetch per eseguire una chiamata POST all'endpoint "movimenti/entrata" passando come body il payload json. Inoltre, settiamo i valori di headers richiesti dal backend per correttamente interpretare il body.

```

49         .then(response => response.json())
50         .then(data => {
51             console.log(data);
52             const msg = `
53                 <ul>
54                     <li><strong>idLocazione:</strong> ${data.idLocazione}</li>
55                     <li><strong>corsia:</strong> ${data.corsia}</li>
56                     <li><strong>colonna:</strong> ${data.colonna}</li>
57                     <li><strong>ripiano:</strong> ${data.ripiano}</li>
58                     <li><strong>idMovimento:</strong> ${data.idMovimento}</li>
59                     <li><strong>codiceInterno:</strong> ${data.codiceInterno}</li>
60                 </ul>
61             `;
62             document.getElementById("message").innerHTML = msg;
63         })
64         .catch(error => {
65             document.getElementById("message").textContent = "Errore: " + error;
66         });
67     });
68 </script>
69 </body>

```

In questo pezzo di codice possiamo trovare i metodi then che vengono attivati all'avvenuta risposta del server.

Il primo then estrae la risposta, il secondo lo elabora e la inserisce dentro ad una costante con un HTML in cui vengono iniettati i valori della risposta, questa costante viene in seguito assegnata all'elemento HTML con id message.

Conclusioni

Questo progetto mi ha permesso di mettere in pratica ed approfondire le nozioni fornitemi durante il corso, in particolare ho avuto occasione di utilizzare le conoscenze di SQL per costruire un modello di persistenza coerente con i requisiti del progetto assegnatomi.

Facendo leva sul linguaggio Java e il suo framework Spring Boot sono stato in grado di costruire delle API che permettono a client di diversa natura di utilizzare le funzionalità logiche di business implementate all'interno dell'API.

Anche il lato di frontend è stato sviluppato utilizzando HTML e JavaScript, questo mi ha permesso di esplorare il legame tra aspetti di layout HTML, aspetti di visualizzazione CSS e aspetti di dinamismo logico JavaScript per fornire all'utente un'interfaccia WEB per interagire con le funzionalità dell'API.

Il progetto per quanto didattico ed incompleto contiene tutti gli elementi basilari di un'applicazione client-server moderna.

A mio parere esistono dell'aree di miglioramento del mio codice e delle funzionalità:

- Accesso all'API solo attraverso ad utenti loggati.
- Aumento delle API aggiungendo metodi di cancellazione e ricerca avanzata attraverso query parameter.
- Gestione dei risultati attraverso paginazione per mantenere costante i tempi di risposta nelle get.
- Aggiunta di una WEB view per lo stato dei movimenti degli articoli presenti nel magazzino.
- Implementare logiche di controllo su validità del barcode, estraendo informazioni applicativa dal valore del barcode stesso
- Implementare client su piattaforma Android.